Lezione 17 - Gli Streams

Introduzione

Le **Collection** rappresentano da sempre la libreria fondamentale di Java per raggruppare, organizzare e rappresentare insiemi di dati in memoria.
Tuttavia, quando si devono eseguire operazioni sui dati (filtri, trasformazioni, ordinamenti, aggregazioni), l'approccio tradizionale è di tipo **programmatico o imperativo**:

- **si definisce *come* eseguire l'operazione**
- **si utilizzano cicli (`for`, `while`), iteratori e variabili temporanee**
- **si gestisce esplicitamente il flusso di controllo**

Questo comporta codice più verboso e focalizzato sui dettagli operativi piuttosto che sull'obiettivo dell'elaborazione.
Esempio:

Java

```java
List<Dish> lowCaloriaDishes = new ArrayList<Dish>();
for (Dish d:menu){
    if(d.getCalories()<400){
        lowcaloriesDishes.add(d);
    }
}
```

Con **Java 8**, viene introdotto il concetto di **Stream**, che fornisce:

- **uno strumento alternativo per eseguire le operazioni più comuni sui dati, adottando un approccio dichiarativo.**

In un approccio dichiarativo:

- **si specifica *cosa* si vuole ottenere**
- **non si descrivono esplicitamente i passi di iterazione**
- **il framework si occupa della gestione del flusso e dell'ottimizzazione**

La sintassi degli Stream ricorda per molti aspetti quella di **SQL**, dove si esprime una query dichiarando il risultato desiderato, senza definire nel dettaglio l'algoritmo di scansione dei dati.

```sql
select *
From Dish
Where calorie <400
```

Java 8 introduce quindi, all'interno della piattaforma **Java SE**, il concetto di **programmazione dichiarativa**, che era già alla base di tecnologie enterprise come gli **EJB** e, più in generale, dei framework moderni. In questi contesti, lo sviluppatore dichiara regole, configurazioni o trasformazioni, lasciando al framework il compito di orchestrare l'esecuzione.

Gli Stream rappresentano quindi un'evoluzione naturale del linguaggio: dopo l'introduzione delle **lambda expressions** e delle **interfacce funzionali**, Java offre ora un modello completo per elaborare collezioni di dati in modo più espressivo, compatto e orientato al risultato.

Nei prossimi punti verrà analizzata la struttura di uno Stream, le sue operazioni intermedie e terminali e il modo in cui si integra con le Collection.

Differenza tra approccio Java 7 e Java 8

Gli **Stream** rappresentano una nuova libreria introdotta in Java 8 per la manipolazione e gestione dei dati in forma **dichiarativa**, utilizzabile insieme o in alternativa alle Collection tradizionali.

L'idea di base è quella di descrivere *cosa* si vuole ottenere da un insieme di dati, lasciando al framework il compito di gestire l'iterazione e l'esecuzione delle operazioni.

Esempio pratico

Dato un **menu** (una `Collection` di oggetti `Dish`), si vogliono recuperare:

- i piatti con meno di 400 calorie
- ordinati per numero di calorie
- ottenendo infine solo i **nomi** dei piatti

Vediamo prima l'approccio tradizionale in **Java 7**.

```java
List<Dish> lowCalorieDishes = new ArrayList<>();
```

```java

for (Dish d : menu) {
    if (d.getCalories() < 400) {
        lowCalorieDishes.add(d);
    }
}

```

In questo primo passaggio:

- si itera esplicitamente sulla collection
- si applica un filtro manuale (`if`)
- si costruisce una nuova lista contenente solo i piatti con meno di 400 calorie

Successivamente, occorre ordinare la lista:

```java
Collections.sort(lowCalorieDishes, new Comparator<Dish>() {
    public int compare(Dish d1, Dish d2) {
        return Integer.compare(d1.getCalories(), d2.getCalories());
    }
});
```

Qui:

- si utilizza `Collections.sort`
- si definisce un `Comparator` anonimo
- si specifica manualmente il criterio di ordinamento

Infine, si estraggono i nomi dei piatti:

```java
List<String> lowCalorieDishNames = new ArrayList<>();

for (Dish d : lowCalorieDishes) {
    lowCalorieDishNames.add(d.getName());
}
```

Anche in questo caso:

- si usa un ulteriore ciclo
- si crea una nuova lista
- si effettua una trasformazione manuale da `Dish` a `String`

📋 Considerazione sull'approccio Java 7

L'intero processo richiede:

- più cicli espliciti
- strutture dati intermedie
- codice verboso
- gestione manuale dell'iterazione

Si tratta di un approccio **imperativo**, in cui si descrivono tutti i passaggi necessari per ottenere il risultato.

Con Java 8, grazie agli **Stream**, la stessa operazione può essere espressa in modo molto più compatto e dichiarativo, descrivendo direttamente la sequenza logica di operazioni:

- filtro
- ordinamento
- trasformazione

Approccio in Java 8

Con **Java 8**, utilizzando i nuovi **Stream** insieme alle **lambda expression**, è possibile riscrivere lo stesso esempio in modo molto più compatto ed espressivo. Versione corretta e completa del codice:

Java

```java
List<String> lowCaloricDishesNames =
    menu.stream()
    .filter(d->d.getCalories<400) // filtra i piatti in base alle calorie minori di 400
    .sorted(d1,d2)->Integer.compare(d1.getCalories(), d2.getCalories()) // ordina in ordine crescente rispetto alle calorie
    .map(d->d.getName()) // estrea i nomi dei piatti
```

```
6        .collect(Collectors.toList()) // raccoglie il risultato in una Collection di tipo List
```

Analisi:

1. `menu.stream()`:
   - **Si crea uno Stream a partire dalla collection `menu`.**
     - `menu` **è la sorgente (source)**
     - `stream()` **produce un flusso di dati su cui applicare operazioni**
     - **in questa fase non viene ancora eseguita alcuna elaborazione**
2. `.filter(d -> d.getCalories() < 400)`
   - **È un'operazione intermedia.**
     - **Accetta un** `Predicate<Dish>`
     - **Per ogni elemento `d` verifica la condizione**
     - **Passano allo step successivo solo i piatti con meno di 400 calorie**

> 🖊 **Concettualmente:** ***filtriamo i piatti meno calorici***

3. `.map(d -> d.getName())`
   - **È un'operazione di trasformazione.**
     - **Accetta una `Function<Dish, String>`**
     - **Trasforma ogni oggetto `Dish` nel suo nome (`String`)**

> 🖊 **Concettualmente:** ***estraiamo solo i nomi dei piatti***

5. `.collect(Collectors.toList())`
   - **l'operazione terminale.**
     - **Converte lo stream in una `List<String>`**
     - **Senza un'operazione terminale, lo stream non viene eseguito**

> 🖊 **Qui la pipeline viene effettivamente valutata.**

L'elemento centrale è il passaggio da un approccio **imperativo** ad un approccio **dichiarativo**:

- **non si descrive più *come* iterare sui dati**

- **non si gestiscono manualmente cicli e strutture intermedie**
- **si dichiara direttamente la sequenza di trasformazioni da applicare**

Un'altra caratteristica fondamentale è la presenza della **pipeline**, cioè una *catena di operazioni* in cui:

- **il risultato di una operazione viene passato direttamente alla successiva**
- **ogni fase svolge un compito specifico (filtro, ordinamento, trasformazione, raccolta)**
- **il flusso dei dati procede in modo lineare, come in una catena di montaggio**

Dal punto di vista concettuale, una pipeline di stream è composta da:

1. **Sorgente (source)** → **ad esempio una collection**
2. **Operazioni intermedie** → **trasformano o filtrano i dati**
3. **Operazione terminale** → **produce il risultato finale**

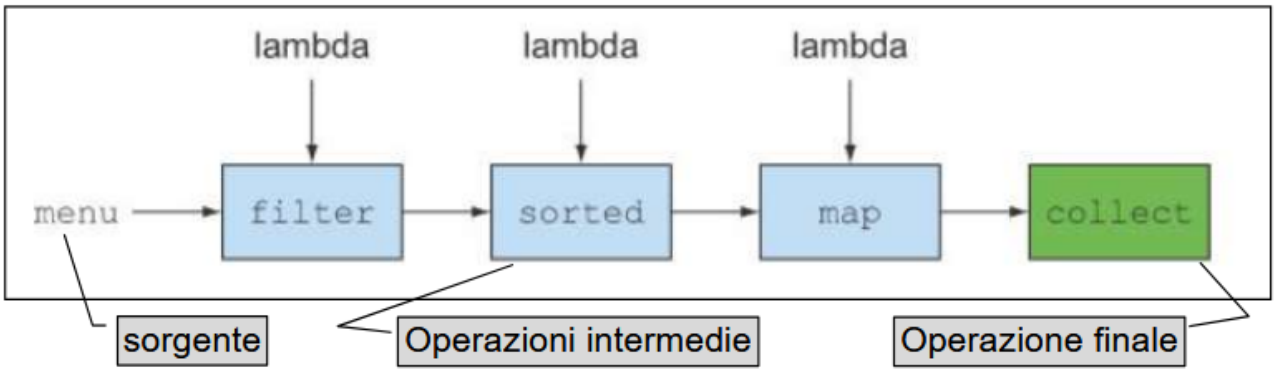
> ✓ **Questo modello rende il codice:**
>
> - più leggibile
> - più vicino alla descrizione del problema
> - meno soggetto a errori legati alla gestione manuale dell'iterazione

Gli Stream rappresentano quindi il completamento naturale delle lambda e delle interfacce funzionali:

- **permettono di applicare concretamente lo stile dichiarativo alla manipolazione delle collezioni.**

Schema della pipeline delle operazioni

Riprendendo l'esempio precedente, possiamo rappresentare la struttura di uno Stream come una **pipeline composta da tre elementi fondamentali**:



Osservando lo schema, il flusso risulta **sequenziale e modulare**:

- ogni fase riceve un insieme di dati in ingresso, li elabora e li passa allo step successivo.

1. **Sorgente:**
   - È il punto di partenza del flusso
   - In questa fase non viene ancora eseguita alcuna elaborazione: si definisce soltanto il flusso di dati su cui operare.

> **La sorgente può essere:**
>
> - **una** `Collection`
> - **un** array
> - **un generatore di dati**
> - **una sorgente I/O**

2. **Operazioni intermedie:**
   - Le operazioni intermedie hanno il compito di **trasformare, filtrare o riorganizzare** i dati provenienti dalla sorgente.:
     - **ricevono uno stream in ingresso**
     - **restituiscono un nuovo stream**
     - **non producono un risultato finale**
     - **sono *lazy* (non vengono eseguite finché non è presente un'operazione terminale)**
   - Inoltre possono essere:
     - **in numero arbitrario**
     - **concatenate liberamente**
     - **organizzate secondo la logica del problema**

✎ **Non esiste un limite al numero di operazioni intermedie:**

è possibile costruire pipeline anche complesse, mantenendo però una struttura lineare e leggibile.

3. **Operazione finale:**
   - **L'operazione finale (o terminale):**
   - **conclude lo stream**
   - **attiva l'esecuzione della pipeline**
   - **produce un risultato concreto**

✎ **Senza un'operazione terminale, lo stream non viene eseguito.**

Caratteristiche principali degli Stream

Possiamo affermare che gli Stream sono:

1️⃣ Dichiarativi

Utilizzano le **espressioni lambda** per descrivere *cosa* fare sui dati, non *come* farlo.
L'iterazione è gestita internamente dal framework, rendendo il codice più compatto e leggibile.

2️⃣ Componibili

Le operazioni possono essere concatenate in modo naturale.

Ogni operazione intermedia:

- restituisce un nuovo Stream
- permette di costruire una pipeline fluida
- può essere combinata con altre operazioni in ordine logico

Questo consente di creare sequenze di trasformazioni senza strutture dati intermedie esplicite.

ⓘ **Queste operazioni intermedie possono essere quante ce ne pare e in un rodine che ci pare**

3️⃣ Parallelizzabili

Gli Stream supportano nativamente l'esecuzione in parallelo.

È possibile sostituire:

```java
menu.stream()
```

con

```java
menu.parallelStream()
```

oppure utilizzare `.parallel()` nella pipeline.

In questo modo:

- l'elaborazione può essere suddivisa su più thread
- il framework gestisce automaticamente la suddivisione dei dati
- non è necessario scrivere codice esplicito per il multithreading

Schema dei passi degli Streams

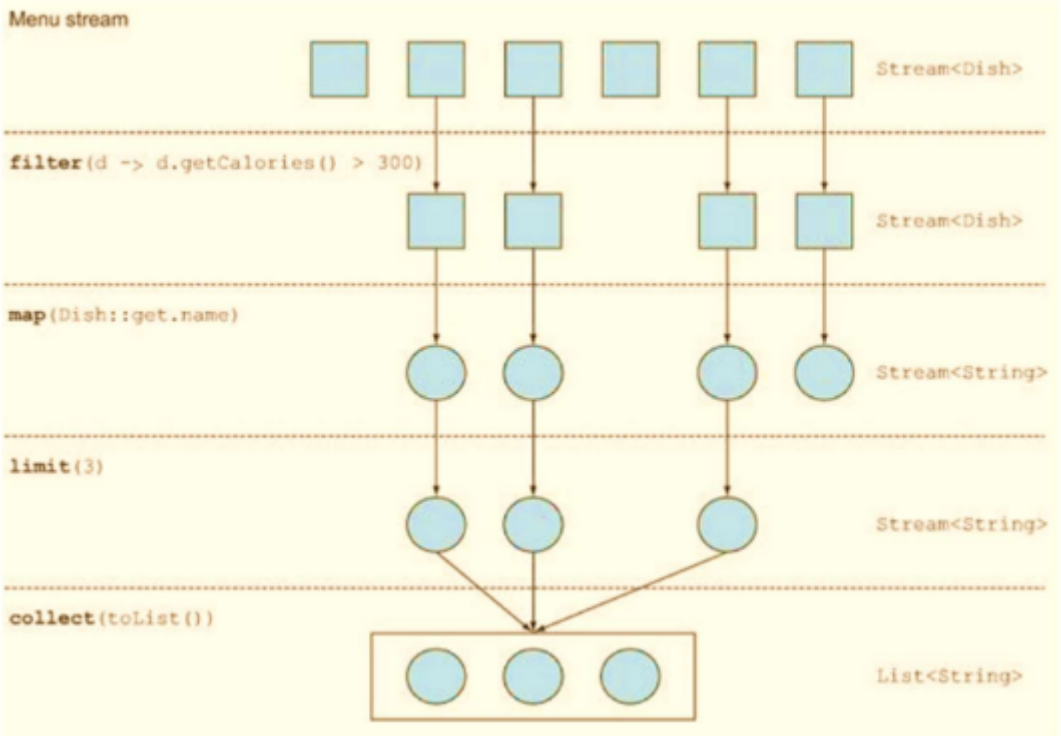
Consideriamo il seguente esempio:

```java
import java.util.stream.Collectors;
import java.util.List;

List<String> threeHighCaloricDishesNames =
    menu.stream()
        .filter(d -> d.getCalories() > 300)
        .map(d -> d.getName())
        .limit(3)
        .collect(Collectors.toList());

System.out.println(threeHighCaloricDishesNames);
```

Questo codice può essere schematizzato secondo questa immagine



Analisi dello schema dell'immagine

L'immagine evidenzia chiaramente la natura **sequenziale e progressiva** dello stream.

1 Menu stream → `Stream<Dish>`

In alto sono rappresentati i singoli elementi della collection `menu`.

Quando viene chiamato:

```java
menu.stream()
```

si ottiene uno `Stream<Dish>`.

Ogni quadrato in alto rappresenta un oggetto `Dish` che entra nel flusso.

2 `filter(d -> d.getCalories() > 300)`

Nel secondo livello dell'immagine si vede che:

- **alcuni elementi vengono "scartati"**
- **solo quelli che soddisfano la condizione proseguono**

Il filtro applica un `Predicate<Dish>`:

```java
Dish -> boolean
```

- **Solo i piatti con calorie maggiori di 300 continuano nel flusso.**

> ⚠ **Importante:**
>
> lo stream non crea una nuova lista intermedia.
> Gli elementi vengono valutati uno alla volta.

3️⃣ `map(Dish::getName)` → `Stream<String>`

Nel livello successivo dell'immagine i quadrati diventano cerchi.

Questo rappresenta il **cambio di tipo dello stream**:

- **prima:** `Stream<Dish>`
- **dopo:** `Stream<String>`

L'operazione `map()` applica una trasformazione:

```java
Dish -> String
```

- **Viene estratto solo il nome del piatto.**

4️⃣ `limit(3)`

Nel passaggio successivo si osserva che:

- **il flusso si interrompe dopo 3 elementi**
- **solo i primi tre che soddisfano i passaggi precedenti vengono mantenuti**

`limit(3)` è un'operazione intermedia **short-circuiting**:

- **interrompe l'elaborazione quando raggiunge il numero richiesto**
- **migliora l'efficienza**

5️⃣ `collect(toList())` → `List<String>`

L'ultimo blocco in basso nell'immagine rappresenta la **raccolta finale**.

`collect()` :

- **è l'operazione terminale**
- **attiva l'intera pipeline**
- **produce una** `List<String>`

> ⏳ **Solo in questo momento lo stream viene effettivamente eseguito.**

Cos'è uno Stream

Uno **Stream** è:

- **un oggetto che rappresenta una sequenza di dati provenienti da una sorgente, sulla quale si intendono applicare una o più operazioni di elaborazione.**

Quando si parla di *sequenza di dati* si intende:

- **un flusso ordinato di elementi, concettualmente simile a quello gestito dalle Collection.**
  Anche gli Stream sono definiti tramite un'interfaccia (`Stream<T>` **) che ne descrive il comportamento generale e le operazioni disponibili.**

> 📋 **Differenza fondamentale tra Collection e Stream**
>
> La distinzione principale è la seguente:
>
> - **Le Collection definiscono strutture dati**
> - **Gli Stream definiscono operazioni sui dati**
>
> Una `List`, una `Set` o una `Map`: **servono a memorizzare e organizzare dati in memoria.**
> Uno Stream, invece, non memorizza dati: **li elabora.**

Concetto di sorgente

Gli Stream non sono strutture dati autonome, **ma oggetti che attingono elementi da una sorgente.**

La sorgente può essere:

- **una** `Collection`
- **un array**
- **un file**
- **una risorsa I/O**
- **un generatore di valori**

Lo Stream utilizza la sorgente per costruire un flusso di elaborazione, ma non modifica mai la sorgente stessa.

Inoltre i dati nello Stream sono sequenziali nel senso che:

- **vengono letti in un ordine definito dalla sorgente**
- **lo Stream non altera l'ordine originale (a meno che non venga applicata un'operazione esplicita come** `sorted()` **)**

In altre parole, lo Stream lavora sui dati senza modificarne la struttura originaria.

Immutabilità della sorgente

Un altro punto importante da ricordare è l'immutabilità della sorgente, ovvero:
Qualunque operazione venga eseguita tramite uno Stream:

- **non modifica la collection originale**
- **non altera i dati nella sorgente**
- **produce eventualmente un nuovo risultato**

> ⚠ **Questo è un punto importante: lo Stream è uno strumento di trasformazione, non di modifica diretta.**

> 📋 **Operazioni supportate**
>
> Gli Stream mettono a disposizione operazioni in stile **SQL-like**, tra cui:
>
> - filtraggio (`filter`)
> - ordinamento (`sorted`)
> - trasformazione (`map`)
> - raggruppamento (`collect`)
> - ricerca (`findFirst` , `anyMatch`)
> - aggregazione (`count` , `reduce`)

Tali operazioni possono essere eseguite:

- in modo **sequenziale**
- oppure in modo **parallelo**, sfruttando il multithreading

Collection vs Stream

Abbiamo già anticipato la differenza tra Collection e Stream:

- rappresentano **due modelli diversi di gestione dei dati** in Java.

1️⃣ Collection

Una Collection è una struttura dati.

Caratteristiche principali

- **Contiene effettivamente i dati in memoria**
- **Tutti gli elementi devono essere presenti prima di poterli elaborare**
- **L'elaborazione avviene sui dati già memorizzati**
- **Uso di iteratore esterno**

Implicazioni

✓ Occupazione di memoria

- **Può essere elevata, perché tutti i dati devono essere caricati**
- Esempio: una `List` con milioni di elementi occupa memoria per tutti gli elementi

✓ Elaborazione immediata

- **Alcune strutture elaborano i dati al momento dell'inserimento/rimozione**
  Esempio:
  - `TreeSet` mantiene l'ordinamento mentre si inseriscono gli elementi

✓ Iterazione esplicita

- Il programmatore controlla l'iterazione:

Java

```
for (String s : list) {
    ...
}
```

2️⃣ Stream

Uno Stream è un flusso di dati, non una struttura dati.

Caratteristiche principali

- **Non memorizza i dati**
- **Legge i dati da una sorgente**
- **Elabora i dati su richiesta (on demand / just-in-time)**
- **Usa iterazione interna**

✓ Occupazione di memoria ridotta

- **I dati non devono essere tutti presenti contemporaneamente**
- **Gli elementi vengono prodotti e consumati progressivamente**

Esempio concettuale:

- Lettura di un file riga per riga
- Generazione di numeri infiniti (`Stream.iterate()`)

✓ Consumabilità

Uno stream può essere consumato **una sola volta**:

Java

```java
Stream<String> s = list.stream();
s.count();      // OK
s.count();      // ERRORE
```

Dopo un'operazione terminale, lo stream è chiuso.

Iterazione interna

Non scriviamo un ciclo esplicito:

Java

```java
list.stream()
    .filter(x -> x > 10)
```

```
        .forEach(System.out::println);
```

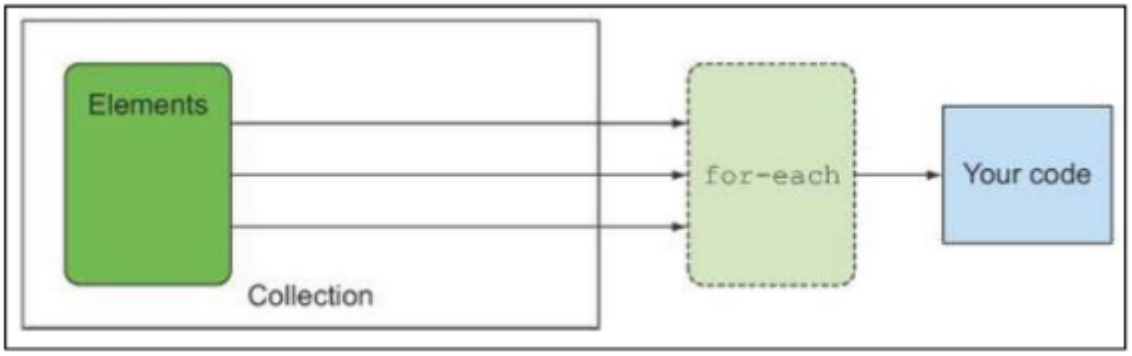
È lo Stream che gestisce l'iterazione.

Iteratore interno vs iteratore esterno

Quindi la differenza tra **Collection** e **Stream** si riflette nel modo in cui avviene l'iterazione sugli elementi.

1 Iteratore esterno (Collection)

Nelle **Collection**, l'iterazione è **controllata esplicitamente dal programmatore**.



Caratteristiche:

- **Il programmatore gestisce il ciclo**
- **Si decide come scorrere gli elementi**
- **Si controlla manualmente il flusso**

  Esempio:

Java

```java
for (String s : list) {
    System.out.println(s);
}
```

Oppure:

Java

```java
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
```

Quindi in questo tipo di approccio, detto *Programmatico (imperativo)* si specificano

- **passi dell'algoritmo**
- **il controllo del ciclo**
- **la gestione dell'iterazione**

In altre parole, **si descrive come fare l'operazione.**

2️⃣ Iteratore interno (Stream)

Invece negli stream l'iterazione è gestita internamente dalla libreria.
Quindi il programmatore non controlla il ciclo, ma definisce **cosa fare** sugli elementi.

Esempio

Java

```java
list.stream()
    .filter(s -> s.length() > 3)
    .forEach(System.out::println);
```

Qui:

- **non esiste un ciclo esplicito**
- **è lo Stream che gestisce l'iterazione**
- **noi definiamo solo le operazioni**

Quindi in questo tipo di approccio, detto *approccio dichiarativo(SQL- like)*, si descrive:

- **cosa si vuole ottenere**
- **non come scorrere i dati**

L'architettura degli Stream

L'interfaccia `java.util.Collection` introduce due metodi **di default** per creare uno Stream a partire da una collezione:

Java

```java
default Stream<E> stream();          // esecuzione sequenziale (single thread)
```

```
2
3   default Stream<E> parallelStream(); // esecuzione parallela
    (multi-thread)
4
```

Il comportamento di default di questi metodi è restituire uno stream che rappresenta la **sequenza degli elementi della collezione chiamante**, senza modificarla.

La differenza tra i due riguarda esclusivamente la modalità di esecuzione:

- `stream()` → **elaborazione sequenziale**
- `parallelStream()` → **elaborazione parallela (sfrutta il framework Fork/Join)**

Creazione di Stream senza Collection

Uno Stream può essere creato anche indipendentemente da una collezione, tramite i metodi statici della classe `Stream`.

Java

```
1   static <T> Stream<T> empty()
2   // → costruisce uno stream vuoto
3
4   static <T> Stream<T> of(T t)
5   // → costruisce uno stream contenente un singolo elemento
6
7   static <T> Stream<T> of(T... values)
8   // → costruisce uno stream contenente 0…n elementi
9
```

Questi metodi permettono di creare stream:

- **a partire da singoli valori**
- **a partire da più valori**
- **oppure uno stream vuoto (utile in logiche condizionali)**

Nota sui metodi di default

I metodi *default* in un'interfaccia (introdotti da Java 8) **sono metodi concreti, cioè con una implementazione già definita nell'interfaccia stessa.**

Permettono di:

- **estendere le interfacce senza rompere la compatibilità con le classi esistenti**
- **fornire un comportamento standard ereditato automaticamente**

Tipi operazioni

Come abbiamo già anticipato le operazioni sugli stream si dividono fondamentalmente in due categorie: **intermedie** e **finali**.

Le **operazioni intermedie** servono a trasformare o filtrare i dati:
ad esempio possiamo filtrare elementi con `filter`, trasformarli con `map`, raggrupparli con `groupingBy`, limitarne il numero con `limit` o ordinarli con `sorted`.
Queste operazioni non producono subito un risultato concreto:

- **concettualmente, preparano lo stream per le fasi successive, creando una pipeline di elaborazione.**
  Una caratteristica importante è che sono **lazy**, cioè:
- **vengono effettivamente eseguite solo quando si invoca un'operazione finale.**
- **In questo modo l'esecuzione è ottimizzata: l'esecutore può combinare o riorganizzare le operazioni intermedie per ridurre il carico computazionale.**

Le **operazioni finali**, invece, sono quelle che:

- chiudono lo stream e generano un risultato concreto.
  Alcuni esempi sono `collect`, per raccogliere i dati in una lista o in un altro contenitore, `reduce`, per combinare gli elementi in un singolo valore, oppure `findFirst` e `anyMatch` per effettuare ricerche.
  Una volta eseguita un'operazione finale, lo stream è considerato **chiuso**:
- **non è più possibile aggiungere altre operazioni, e tentare di farlo genera un'eccezione** `java.lang.IllegalStateException: stream has already been operated upon or closed`.

In sintesi

le operazioni intermedie definiscono:

- ***cosa* vogliamo fare sui dati,**
  ma vengono realmente eseguite solo al momento dell'operazione finale, che invece:
- **produce il risultato concreto e chiude lo stream.**

Di seguito una panoramica delle principali operazioni intermedie, con la descrizione, la chiamata lambda tipica, gli argomenti richiesti e il tipo di ritorno:

Operazione	Descrizione	Chiamata lambda	Argomenti	Tipo di ritorno
`filter`	**Filtra gli elementi**	`T -> boolean`	`Predicate<T>`	`Stream<`
`map`	**Trasforma gli eleme**nti	`T -> R`	`Function<T,R>`	`Stream<`
`flatMap`	**Appiattisce uno stream di strea**m	`T -> Stream<R>`	`Function<T,Stream<R>>`	`Stream<`
`limit`	**Limita il numero di elementi**	—	`long`	`Stream<`
`sorted`	**Ordina gli elementi**	`(T,T) -> int`	`Comparator<T>`	`Stream<`
`distinct`	**Rimuove duplicati**	—	—	`Stream<`
`peek`	**Permette di eseguire un'azione su ogni elemento senza modificare lo stream**	`(T) -> void`	`Consumer<? super T>`	`Stream<`
`skip`	**Salta i primi N elementi**	—	`long`	`Stream<`

Ora andiamo ad esaminare uno per uno questi metodi

Operazione intermedia: `filter`

Tra le operazioni intermedie, il **filtraggio** è senza dubbio una delle più comuni. **Permette di selezionare solo gli elementi di uno stream che soddisfano determinati criteri, definiti tramite un predicato.**
La classe `Stream` mette a disposizione due metodi principali per questo scopo:

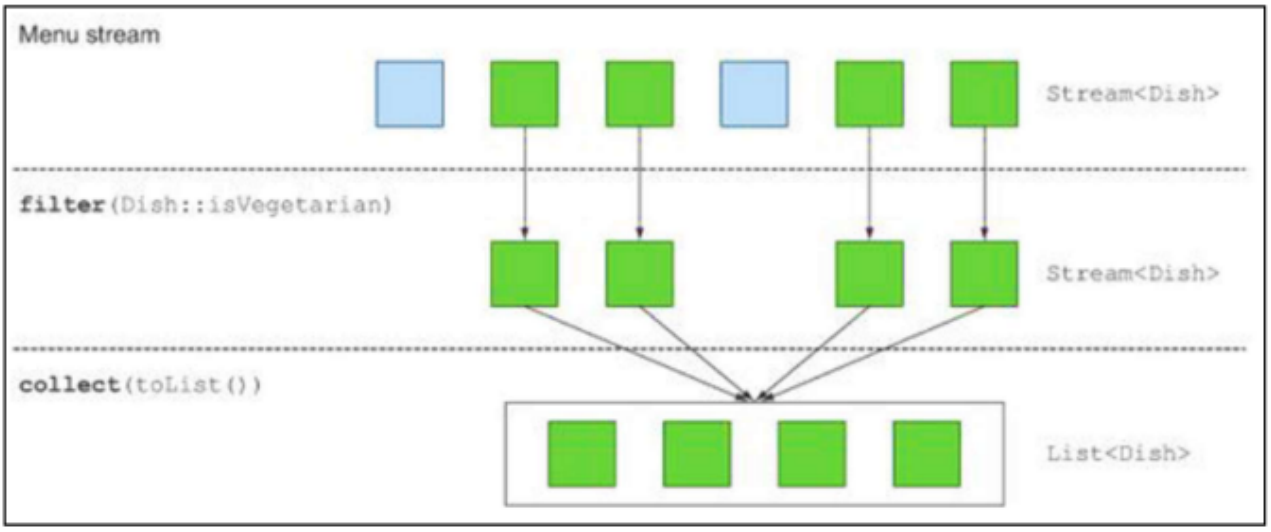
- `Stream<T> filter(Predicate<? super T> predicate)`
  Restituisce uno stream i cui elementi corrispondono al predicato fornito, scartando tutti gli altri.
- `Stream<T> distinct()`
  Restituisce uno stream contenente solo elementi unici, eliminando eventuali duplicati in accordo con il metodo `equals()` degli oggetti.

Quando si applica `filter`, il risultato è un nuovo stream che contiene esclusivamente gli elementi selezionati.
In questo modo è possibile concatenare ulteriori operazioni intermedie o, infine, applicare un'operazione finale per ottenere un risultato concreto.

Ad esempio, se si volesse ottenere un menu vegetariano a partire da una lista di piatti `menu`, si potrebbe scrivere:

Java

```java
List<Dish> vegetarianMenu = menu.stream()
                                .filter(d -> d.isVegetarian())
                                .collect(Collectors.toList());
```



Seguendo questo schema possiamo notare che:

1. la sorgente `menu` inizia la trasmissione,

2. ogni elemento dello stream di tipo `<Dish>` viene filtrato controllando il metodo `isVegetarian()`,
3. e infine gli elementi selezionati vengono raccolti in una nuova lista tramite l'operazione finale `collect`.

Operazione intermedia: `distinct`

Un'altra operazione intermedia molto utile è `distinct`:

- **che permette di eliminare i duplicati dallo stream.**
  Questo è particolarmente comodo quando si vogliono ottenere solo elementi unici, ad esempio dopo un filtraggio o una trasformazione.

Il metodo `distinct()`:

- **restituisce un nuovo stream in cui ogni elemento è presente una sola volta, secondo la definizione di uguaglianza fornita dal metodo `equals()` degli oggetti.**

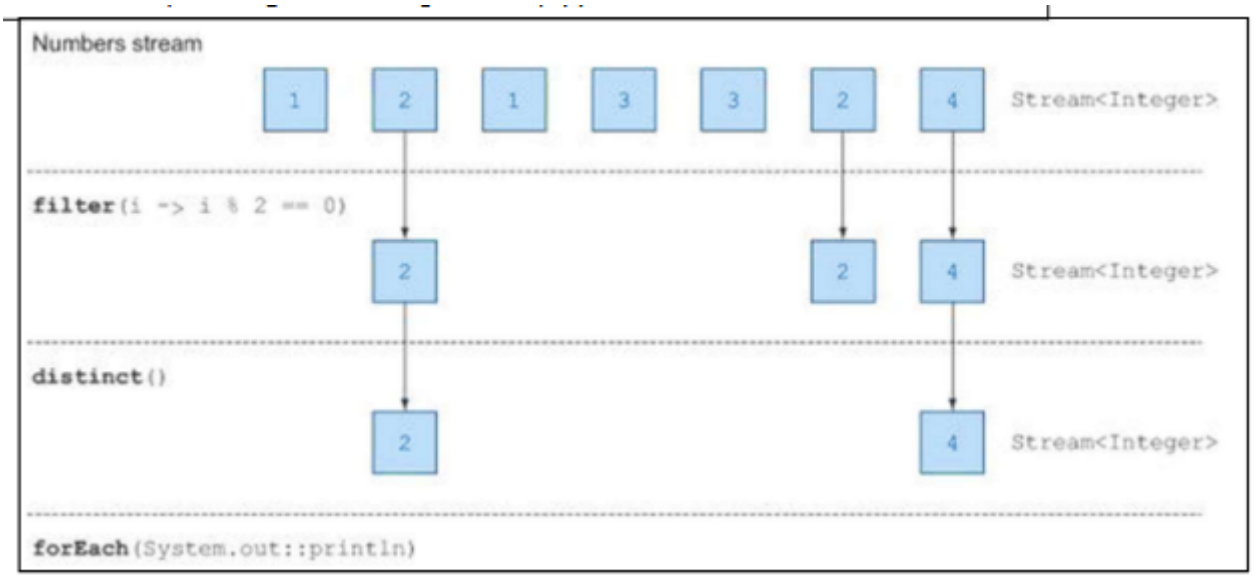
Ad esempio, consideriamo una lista di numeri interi con duplicati:

Java

```java
List<Integer> numbers = Arrays.asList(1, 2, 1, 3, 3, 2, 4);

numbers.stream()
        .filter(i -> i % 2 == 0) // seleziona solo i numeri pari
        .distinct()              // elimina i duplicati
        .forEach(i -> System.out.println(i));

```



Seguendo lo schema di questo stream:

1. **Lo stream parte dalla lista `numbers`.**
2. **L'operazione `filter` seleziona solo i numeri pari.**
3. **L'operazione `distinct` elimina i valori duplicati tra i numeri pari.**
4. **Infine, l'operazione finale `forEach` stampa ogni numero unico rimasto nello stream.**
   L'output quindi sarà:

```
2
4
```

Così come con `filter`, anche `distinct` **crea un nuovo stream pronto per ulteriori operazioni intermedie o per essere chiuso da un'operazione finale.**

Operazione intermedia: `limit`

Un'altra operazione intermedia utile è `limit`, che:

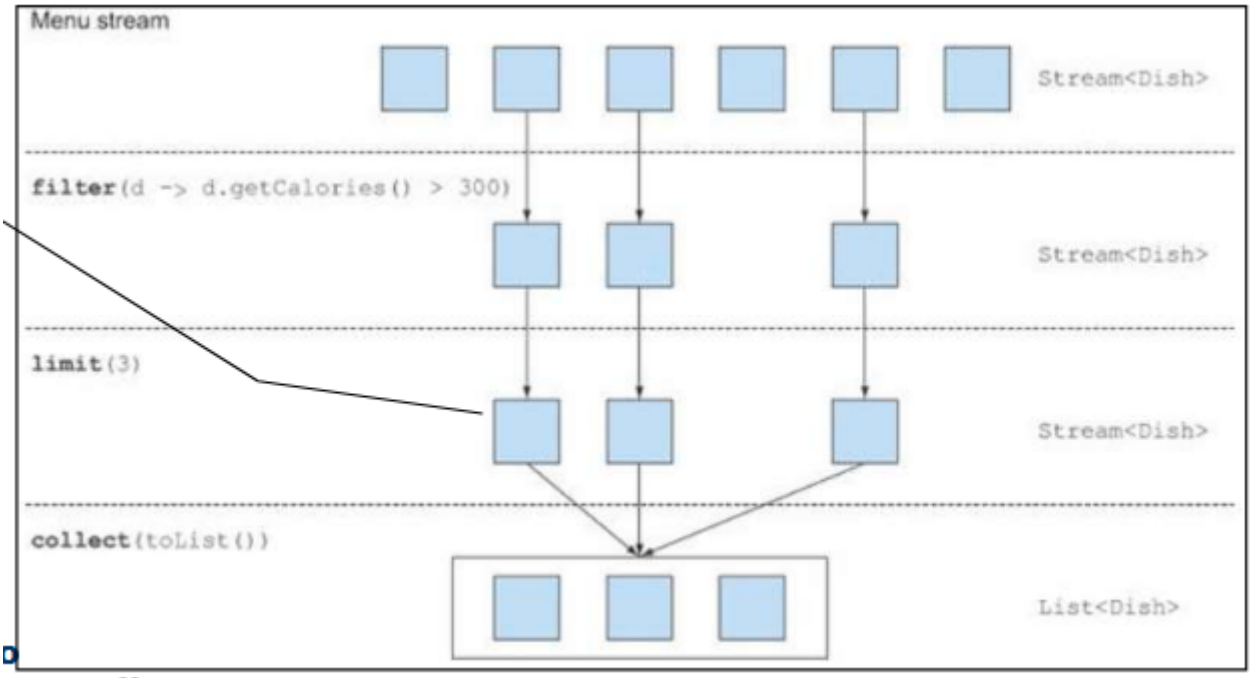
- **permette di limitare il numero di elementi presenti nello stream.**
  Questo può essere comodo quando **si vogliono elaborare solo i primi `N` elementi che soddisfano una certa condizione, senza dover scorrere l'intera sorgente.**

Ad esempio, supponiamo di voler ottenere al massimo tre piatti con più di 300 calorie da una lista `menu`:

Java

```java
List<Dish> dishes = menu.stream()
```

```
                                .filter(d -> d.getCalories() > 300)
                                .limit(3)
                                .collect(Collectors.toList());
```



In questo esempio:

1. **Lo stream parte dalla lista `menu`.**
2. **L'operazione `filter` seleziona solo i piatti con più di 300 calorie.**
3. `limit(3)` **fa sì che lo stream si fermi non appena tre elementi soddisfano il filtro.**
4. L'operazione finale `collect` **raccoglie i piatti filtrati in una lista.**

> ⚠ **Nota importante:**
>
> `limit` è molto performante perché, grazie alla valutazione lazy degli stream, **non è necessario processare tutti gli elementi della sorgente**; **l'operazione termina non appena il numero richiesto di elementi è disponibile.**
> Inoltre, **gli stream rispettano l'ordine definito dalla sorgente, quindi i primi N elementi saranno sempre quelli che compaiono per primi nella lista originale.**

Operazione intermedia: `skip`

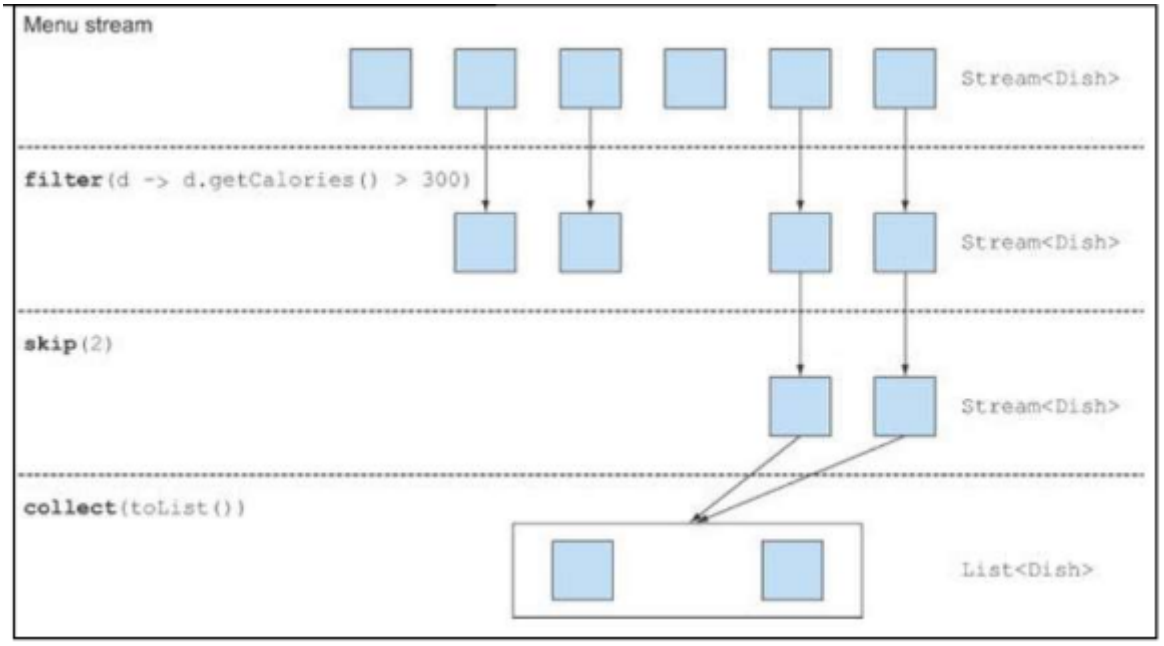
L'operazione `skip` permette di:

- **scartare un certo numero di elementi all'inizio dello stream.**
  È particolarmente utile quando **si vogliono saltare i primi N elementi che soddisfano una condizione**,
  ad esempio per implementare paginazione o selezioni parziali.

Ad esempio, supponiamo di voler ignorare i primi due piatti con più di 300 calorie da una lista `menu`:

Java

```java
List<Dish> dishes = menu.stream()
                        .filter(d -> d.getCalories() > 300)
                        .skip(2)
                        .collect(Collectors.toList());
```



In questo esempio:

1. Lo stream parte dalla lista `menu`.
2. L'operazione `filter` **seleziona solo i piatti con più di 300 calorie.**
3. `skip(2)` **scarta i primi due elementi filtrati.**
4. L'operazione finale `collect` **raccoglie il resto degli elementi in una lista.**

Come per `limit`, anche `skip`:

- rispetta l'ordine degli elementi della sorgente e beneficia della valutazione **lazy:**
  - **gli elementi vengono scartati man mano che lo stream viene processato, senza dover caricare tutti gli elementi in memoria.**

Operazione intermedia: `map`

Un'operazione molto comune sugli stream è quella di:

- **selezionare o trasformare alcuni campi degli oggetti presenti nello stream.**
  Questo corrisponde, a livello concettuale, alla clausola `SELECT` di una query SQL.

Il metodo `map()` permette di:

- **trasformare ogni elemento dello stream in un altro oggetto, applicando una funzione a ciascun elemento.**
  Ad esempio, supponiamo di voler ottenere solo i nomi dei piatti presenti nella lista `menu`:

Java

```java
List<String> dishNames = menu.stream()
                              .map(d -> d.getName()) // invoca getName su ogni elemento Dish
                              .collect(Collectors.toList());
```

In questo caso:

1. Lo stream parte dalla lista `menu`.
2. `map(d -> d.getName())` **trasforma ogni oggetto** `Dish` **in una** `String` **corrispondente al nome del piatto.**
3. L'operazione finale `collect` **raccoglie tutti i nomi in una** `List<String>`**.**

Il metodo `map` può essere **chiamato a cascata** nella pipeline, applicando più trasformazioni successive.
Ad esempio, se volessimo ottenere la **lunghezza dei nomi dei piatti**, potremmo scrivere:

Java

```java
List<Integer> dishNamesLengths = menu.stream()
                                      .map(d -> d.getName()) // ottiene il nome
                                      .map(name -> name.length()) // calcola la lunghezza
```

```
4
    .collect(Collectors.toList());
```

Qui:

- **Ogni `Dish` viene trasformato in una `String` (nome del piatto).**
- **Successivamente, ogni `String` viene trasformata in un `Integer` (lunghezza del nome).**
- **Il risultato finale è una lista di interi con le lunghezze dei nomi dei piatti.**

In questo modo, `map` consente di:

- **estrarre, trasformare e combinare i dati in maniera molto flessibile, sempre mantenendo la catena di invocazioni tipica degli stream.**

Operazione intermedia: `flatMap`

Una piccola variazione sul tema della trasformazione è l'operazione `flatMap`. Questo metodo è particolarmente utile quando si ha:

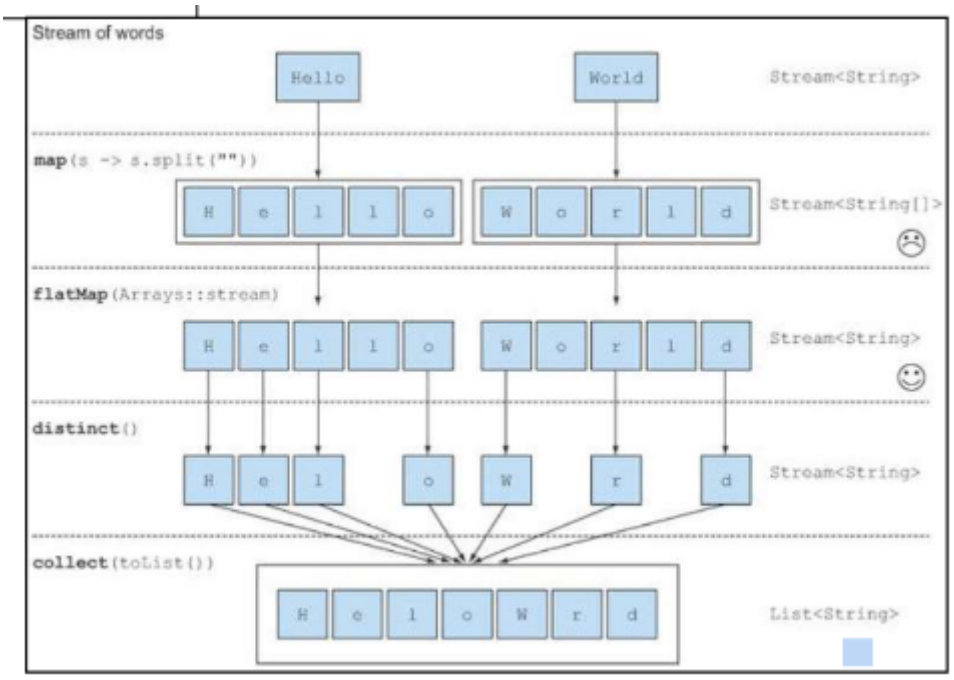
- **uno stream di strutture dati complesse (come array o liste) e si vuole ottenere uno stream dei singoli elementi contenuti in queste strutture, "appiattendo" quindi la gerarchia.**

Prendiamo come esempio il caso precedente in cui avevamo una lista di parole: supponiamo di voler ottenere un elenco dei **diversi caratteri presenti in tutte le parole**, eliminando i duplicati.

Se provassimo a usare solo `distinct()` dopo uno `split`, il risultato non sarebbe quello desiderato:

Java

```java
List<String[]> splitWords = menu.stream()
                                .map(word -> word.split(""))
                                .distinct()
                                .collect(Collectors.toList());
```



- Qui ogni elemento dello stream è un array di `String` (`String[]`).
- L'operazione `distinct` v **aluterebbe l'uguaglianza degli array stessi, non dei singoli elementi al loro interno, quindi non otteniamo una lista di caratteri unici.**

Il metodo `flatMap` risolve questo problema:

```java
List<String> uniqueChars = menu.stream()
                                .map(word -> word.split(""))  // ottiene array di caratteri
                                .flatMap(Arrays::stream)      // fonde tutti gli array in un unico stream di String
                                .distinct()                   // elimina i duplicati
                                .collect(Collectors.toList());
```

- `flatMap(Arrays::stream)` **trasforma uno stream di array in uno stream degli elementi contenuti negli array, mantenendo l'ordine originale.**
- In questo modo, `distinct()` **può lavorare correttamente sui singoli caratteri, producendo una lista di valori unici.**

Firma dei metodi flatMap

I metodi principali associati a `flatMap` sono:

Metodo	Descrizione
`<R> Stream<R> flatMap(Function<? super T, ? extends Stream<? extends R>> mapper)`	**Converte uno stream di strutture in uno stream di elementi, mantenendo il tipo generico R.**
`IntStream flatMapToInt(Function<? super T, ? extends IntStream> mapper)`	**Versione per stream di interi.**
`LongStream flatMapToLong(Function<? super T, ? extends LongStream> mapper)`	**Versione per stream di long.**
`DoubleStream flatMapToDouble(Function<? super T, ? extends DoubleStream> mapper)`	**Versione per stream di double.**

⚠ Note importanti:

- `flatMap` è un'operazione intermedia che **ritorna sempre uno stream**, pronto per ulteriori operazioni.
- Fonde strutture come array o liste in un unico stream, quindi **due invocazioni consecutive di flatMap senza intervenire con un'operazione finale possono portare a errori di compilazione o risultati non voluti**.

☰ In sintesi:

`flatMap` è essenziale quando vogliamo "appiattire" dati strutturati in stream complessi, permettendo di continuare a usare tutte le operazioni intermedie e finali sugli elementi singoli.

Operazione intermedia: `peek`

Il metodo `peek` è:

- **uno strumento utile principalmente a scopo di test o debug.**
- **Permette di osservare o applicare un comportamento su ogni elemento dello stream senza modificarlo, restituendo sempre un nuovo stream pronto per ulteriori operazioni.**

La firma del metodo è:

```java
Stream<T> peek(Consumer<? super T> action)
```

Esempio pratico:

```java
List<String> list = new ArrayList<>();
list.add("one");
list.add("two");
list.add("three");
list.add("four");

List<String> filterList = list.stream()
                              .filter(e -> e.length() > 3)
                              .peek(e -> System.out.println("Filtered value: " + e))
                              .map(e -> e.toUpperCase())
                              .peek(e -> System.out.println("Mapped value: " + e))
                              .collect(Collectors.toList());
```

- `peek` **stampa i valori intermedi degli elementi mentre scorrono nella pipeline**.
- Nel caso sopra, **i valori filtrati (`three`, `four`) e quelli trasformati in maiuscolo (`THREE`, `FOUR`) vengono stampati, ma la lista finale contiene solo `[THREE, FOUR]`.**
- È uno strumento molto comodo per **monitorare il flusso dei dati** senza interrompere o modificare lo stream.

Operazione intermedia: `sorted`

L'operazione `sorted` **permette di ordinare gli elementi dello stream secondo un criterio definito, oppure secondo l'ordinamento naturale degli elementi.**

I metodi principali sono:

- `Stream<T> sorted()` → **ordina secondo l'ordinamento naturale degli elementi.**

- `Stream<T> sorted(Comparator<? super T> comparator)` → **ordina secondo un comparatore specificato.**

Esempio pratico:

Java

```java
List<String> list = new ArrayList<>();
list.add("one");
list.add("two");
list.add("three");
list.add("four");

list.stream()
    .peek(e -> System.out.println("Original value: " + e))
    .sorted((s1, s2) -> s1.compareTo(s2))  // oppure .sorted() per l'ordinamento naturale
    .peek(e -> System.out.println("Sorted value: " + e))
    .collect(Collectors.toList());
```

- La prima `peek` **mostra i valori originali prima dell'ordinamento.**
- La seconda `peek` **mostra i valori dopo l'ordinamento, ad esempio** `[four, one, three, two]`**.**
- `sorted` è quindi **utile per riorganizzare gli elementi prima di applicare operazioni finali come** `collect`**.**

Operazioni finali sugli Stream

Mentre le operazioni intermedie servono a trasformare o filtrare i dati, le **operazioni finali** sono quelle che:

- **chiudono lo stream e producono un risultato concreto.**
- **Possono restituire valori singoli, aggregazioni o collezioni, e terminano la pipeline di elaborazione.**

Di seguito una panoramica delle operazioni finali più comuni, con la chiamata lambda tipica, gli argomenti richiesti e il tipo di ritorno:

Operazione	Descrizione	Chiamata lambda	Argomenti	Tipo di ritorn
`anyMatch`	**Controlla se almeno un**	`T -> boolean`	`Predicate<T>`	`boolean`

Operazione	Descrizione	Chiamata lambda	Argomenti	Tipo di ritorn
	elemento soddisfa il predicato			
`allMatch`	Controlla se tutti gli elementi soddisfano il predicato	`T -> boolean`	`Predicate<T>`	`boolean`
`noneMatch`	Controlla se nessun elemento soddisfa il predicato	`T -> boolean`	`Predicate<T>`	`boolean`
`findAny`	Restituisce un elemento qualsiasi dello stream	—	—	`Optional<T>`
`findFirst`	Restituisce il primo elemento dello stream	—	—	`Optional<T>`
`forEach`	Applica un'azione a ogni elemento	`T -> void`	`Consumer<? super T>`	`void`
`collect`	Raccoglie i dati in una collezione o altro contenitore	—	`Collector<T,A,R>`	`R`
`reduce`	Combina gli elementi in un singolo risultato	`(T,T) -> T`	`BinaryOperator<T>`	`Optional<T>`
`count`	Conta gli elementi	—	—	`long`
`max`	Trova il massimo	`(T,T) -> int`	`Comparator<T>`	`Optional<T>`

Operazione	Descrizione	Chiamata lambda	Argomenti	Tipo di ritorn
	secondo un comparatore			
`min`	Trova il minimo secondo un comparatore	`(T,T) -> int`	`Comparator<T>`	`Optional<T>`
`average`	Calcola la media	—	—	`OptionalDou`

Operazione finale: stoccaggio dei dati

Una delle operazioni finali più comuni è **raccogliere i dati in una collezione**, ad esempio una `List`, dopo aver applicato tutte le operazioni intermedie della pipeline.

Gli stream offrono il metodo `collect()` a questo scopo, con due versioni principali:

Java

```java
<R,A> R collect(Collector<? super T,A,R> collector)
<R> R collect(Supplier<R> supplier,
              BiConsumer<R,? super T> accumulator,
              BiConsumer<R,R> combiner)
```

Esempio pratico:

Java

```java
List<Integer> dishNameLengths = menu.stream()
                                    .map(d -> d.getName())   // seleziona il nome del piatto
                                    .map(s -> s.length())    // calcola la lunghezza del nome
                                    .collect(Collectors.toList()); // raccoglie i risultati in una lista
```

- Lo stream parte dalla lista `menu`.

- Le operazioni intermedie trasformano ogni `Dish` in una `String` (nome) e poi in un `Integer` (lunghezza).
- L'operazione finale `collect` raccoglie tutti gli elementi elaborati in una **nuova lista**.

In questo modo, `collect` consente di **concretizzare il risultato della pipeline** e chiudere lo stream, pronto per l'uso successivo.

Operazioni finale: ricerca

Operazione finale: ricerca con `xxxMatch()`

Tra le operazioni finali più comuni ci sono quelle di **ricerca** tramite i metodi `anyMatch()`, `allMatch()` e `noneMatch()`.
Questi metodi permettono di verificare se uno stream contiene elementi che soddisfano un certo criterio:

- `boolean anyMatch(Predicate<? super T> predicate)` → **restituisce** `true` **se almeno un elemento soddisfa il predicato.**
- `boolean allMatch(Predicate<? super T> predicate)` → **restituisce** `true` **se tutti gli elementi soddisfano il predicato.**
- `boolean noneMatch(Predicate<? super T> predicate)` → **restituisce** `true` **se nessun elemento soddisfa il predicato.**

> ⚠ **Nota importante:**
>
> questi metodi restituiscono solo un booleano, quindi **non consentono di sapere quali elementi corrispondono al criterio**.

Esempi pratici:

Java

```java
if(menu.stream().anyMatch(d -> d.isVegetarian())) {
    System.out.println("The menu is (somewhat) vegetarian friendly!!");
}

if(menu.stream().allMatch(d -> d.getCalories() < 1000)) {
    System.out.println("Yeah! All Dishes are for me!");
}
```

```java

if(menu.stream().noneMatch(d -> d.getCalories() >= 1000)) {
    System.out.println("Yeah! All Dishes are for me!");
}
```

- `anyMatch` verifica se almeno una pietanza è vegetariana.
- `allMatch` verifica se tutte le pietanze hanno meno di 1000 calorie.
- `noneMatch` verifica se nessuna pietanza supera le 1000 calorie.

Tutte queste operazioni sono **cortocircuitate**, cioè interrompono la scansione dello stream non appena il risultato è noto.

Operazione finale: ricerca con `xxxFind()`

Se invece vogliamo **ottenere effettivamente gli elementi** che soddisfano un criterio, possiamo usare i metodi `findFirst()` e `findAny()`.

- `Optional<T> findFirst()` → **restituisce il primo elemento che corrisponde al criterio. Comportamento deterministico.**
- `Optional<T> findAny()` → **restituisce uno qualsiasi degli elementi che corrispondono al criterio.**
    - Comportamento non deterministico, utile per stream paralleli.

Anche questi metodi sono **cortocircuitati**, quindi una volta trovato un elemento non vengono esaminati gli altri.

Esempi pratici:

Java

```java
Optional<Dish> o = menu.stream()
                        .filter(d -> d.isVegetarian())
                        .findAny();  // ritorna una pietanza vegetariana qualsiasi

Dish d1 = menu.stream().findAny().get();     // ritorna una pietanza qualsiasi
Dish d2 = menu.stream().findFirst().get();   // ritorna la prima pietanza
```

Nota sull'uso di `Optional<T>`

- `Optional<T>` è stato introdotto in Java 8 per gestire il ritorno di metodi che **potrebbero non produrre alcun risultato**, evitando l'uso di `null`.
- Un oggetto `Optional` può **contenere un valore oppure essere vuoto**.
- Per verificare se il valore è presente si usa `isPresent()`, mentre `get()` restituisce il valore se presente.

Con questo approccio, le operazioni di ricerca diventano **sicure e chiare**, evitando eccezioni dovute a riferimenti nulli.

Operazione finale: riduzione con `reduce()`

Un'altra operazione finale molto importante è la **riduzione**, che consente di:

- **ricavare un singolo valore a partire da tutti gli elementi dello stream. Questo è simile alle operazioni di aggregazione in SQL, come `SUM`, `AVG` o `MAX`.**

La riduzione funziona applicando **iterativamente una funzione binaria sugli elementi dello stream, accumulando il risultato passo passo, come farebbe un ciclo `for`.**

Metodi principali di `reduce()`

Gli stream offrono tre versioni principali del metodo `reduce`:

1. `T reduce(T identity, BinaryOperator<T> accumulator)`
   - **L'accumulatore parte da un valore iniziale (`identity`) e somma o combina gli elementi dello stream.**
2. `Optional<T> reduce(BinaryOperator<T> accumulator)`
   - **Non prevede un valore iniziale; se lo stream è vuoto, il risultato sarà un `Optional` vuoto.**
3. `<U> U reduce(U identity, BiFunction<U, ? super T, U> accumulator, BinaryOperator<U> combiner)`
   - **Versione più generica che permette di ridurre elementi di tipo diverso o di combinare risultati in stream paralleli.**

Esempi pratici

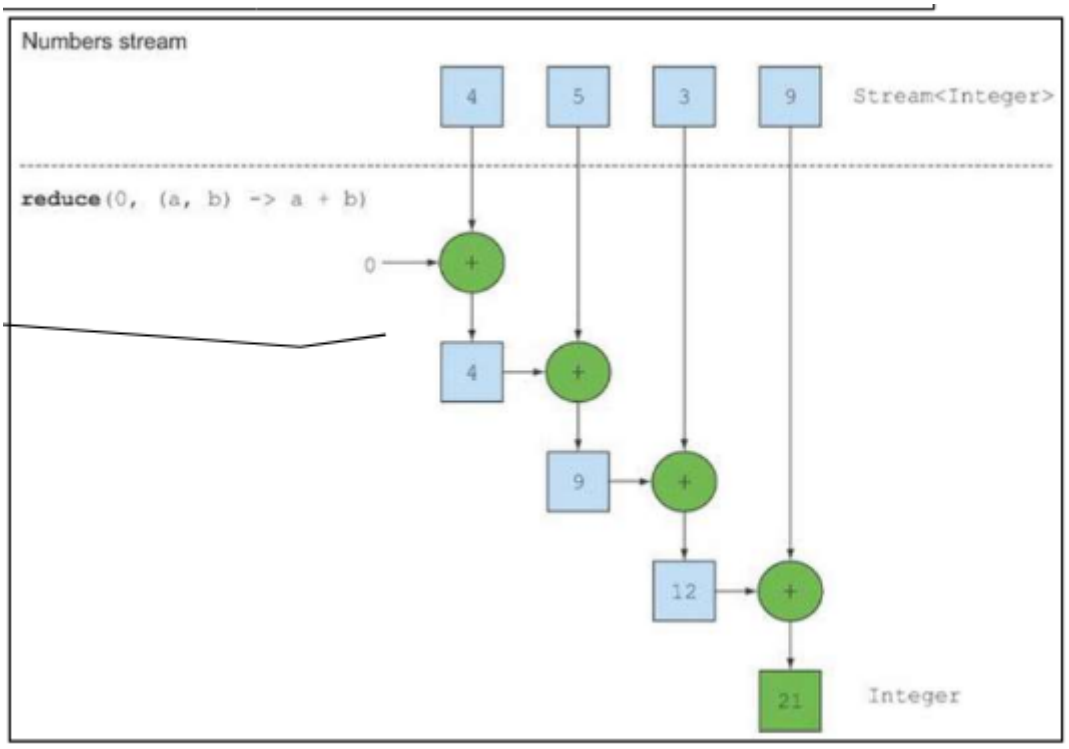
Sommatoria dei numeri di uno stream:

```java
List<Integer> numbers = Arrays.asList(4, 5, 3, 9);

// Con valore iniziale
int sum = numbers.stream()
                 .reduce(0, (a, b) -> a + b);

// Alternativa usando Integer.sum
int sum2 = numbers.stream()
                  .reduce(0, Integer::sum);

// Senza valore iniziale (ritorna Optional)
Optional<Integer> sumOptional = numbers.stream()
                                       .reduce((a, b) -> a + b);
```



Come funziona:

- Il primo parametro (quando presente) rappresenta il **valore accumulato iniziale**.
- L'accumulatore `(a, b) -> a + b` **prende due argomenti: il valore accumulato finora e il prossimo elemento dello stream, e restituisce il nuovo valore accumulato.**
- Se non viene fornito un valore iniziale, lo stream deve avere almeno un elemento; altrimenti il risultato sarà un `Optional.empty()`.

La riduzione è molto flessibile e può essere utilizzata non solo per somme, ma anche per **prodotti, concatenazioni, massimo, minimo** o qualsiasi altra operazione che combini elementi in un singolo valore.

Nuovi metodi per `Number`

Per rendere più agevole l'utilizzo dell'operazione di **riduzione** (`reduce()`), Java ha introdotto alcuni metodi di supporto direttamente nelle classi wrapper numeriche, cioè nelle classi che estendono `Number` come `Short` , `Integer` , `Long` , `Float` e `Double` .

In particolare, sono disponibili metodi statici che rappresentano operazioni elementari molto comuni nelle riduzioni:

- `static int min(int a, int b)`
- `static int max(int a, int b)`
- `static int sum(int a, int b)`

Questi metodi risultano particolarmente utili quando si utilizza `reduce` , perché:

- **permettono di esprimere l'operazione di accumulazione in modo più leggibile e semantico.**
  Ad esempio, invece di scrivere una lambda come `(a, b) -> a + b` , è possibile usare direttamente `Integer.sum(a, b)` , migliorando chiarezza e intenzione del codice.

In sostanza, queste utility semplificano le operazioni di aggregazione numerica all'interno di uno stream.

Operazioni finali: `count` , `max` , `min`

Tra le operazioni finali degli stream, alcune rappresentano casi particolari di riduzione, già pronte all'uso e molto frequenti nella pratica.

`count()`

Java

```java
long count()
```

Questo metodo restituisce il numero totale di elementi presenti nello stream.
È una forma di riduzione che non richiede alcuna funzione di accumulo esplicita:

il framework si occupa internamente del conteggio.

Esempio:

Dato il seguente codice :

```java
List<String> lista = Arrays.asList("", "Red", "", "Green", "Black");
long val = lista.stream()
    .filter(n -> !n.isEmpty())
    .count();
System.out.println(val)
```

Stampa l numero di elementi non vuoti dello stream, cioè 3.

`max()` e `min()`

```java
Optional<T> max(Comparator<? super T> comparator)
Optional<T> min(Comparator<? super T> comparator)
```

Questi metodi permettono di individuare rispettivamente il valore massimo o minimo nello stream, secondo il criterio definito da un `Comparator`.

Il confronto non è implicito (a meno che non si utilizzi uno stream di oggetti che implementano `Comparable` e si sfrutti `sorted()` prima), ma viene determinato esplicitamente dal comparatore passato come argomento.

Il tipo di ritorno è `Optional<T>`, e questo è un aspetto importante:

- **se lo stream è vuoto, il risultato non è `null`, ma un `Optional` vuoto (`Optional.empty()`).**

Questo evita il rischio di `NullPointerException` e obbliga a gestire in modo consapevole l'eventuale assenza di risultato.

> **Nota su `average()`**
>
> Esiste anche il metodo `average()`, **ma non appartiene all'interfaccia `Stream<T>` generica.**
> È disponibile solo nelle versioni specializzate degli stream numerici, come:
>
> - `IntStream`

- `LongStream`
- `DoubleStream`

Anche in questo caso il valore restituito è un `Optional` **specializzato (ad esempio `OptionalDouble`), proprio perché lo stream potrebbe essere vuoto.**

In conclusione:

queste operazioni rappresentano forme di riduzione già strutturate e ottimizzate, che consentono di ottenere risultati aggregati in modo conciso, leggibile e sicuro, evitando di dover implementare manualmente la logica con `reduce()` quando non necessario.

Operazione finale: `forEach`

Un'altra operazione finale molto utilizzata è `forEach()`, che consente di applicare un determinato comportamento a **tutti gli elementi** dello stream.

La firma del metodo è:

Java

```java
void forEach(Consumer<? super T> action)
```

Il parametro è un `Consumer`, **quindi una funzione che accetta un elemento e non restituisce alcun valore.**
Si tratta quindi di un'operazione con effetto collaterale (ad esempio stampa, aggiornamento di stato esterno, logging, ecc.).

Aspetto fondamentale: l'ordine

Un punto critico da comprendere riguarda l'**ordine di esecuzione**.

- **L'ordine non è garantito, né con `stream()` né con `parallelStream()`.**
- In particolare, con `parallelStream()` **l'ordine con cui la funzione viene applicata agli elementi è non deterministico.**

Questo significa che, se lo stream è parallelo, **le operazioni possono essere eseguite su thread diversi e in un ordine differente rispetto alla sequenza originale dei dati.**

`forEachOrdered()`

Per mantenere l'ordine definito dalla sorgente dello stream, è disponibile:

Java

```java
void forEachOrdered(Consumer<? super T> action)
```

Con questo metodo:

- **l'ordine è garantito,**
- **le azioni vengono eseguite una dopo l'altra,**
- **non vi è sovrapposizione tra le operazioni.**

Tuttavia, questa garanzia comporta un costo:

- `forEachOrdered()` **è meno performante, specialmente nel caso di stream paralleli, perché limita le possibilità di esecuzione concorrente.**

In sintesi:

- usare `forEach()` **quando l'ordine non è rilevante e si desidera massimizzare le prestazioni;**
- usare `forEachOrdered()` **quando l'ordine logico dei dati è significativo e deve essere preservato.**

È importante ricordare che `forEach` **è un'operazione finale e quindi chiude la pipeline, attivando l'esecuzione delle operazioni intermedie definite in precedenza.**

Specializzazioni degli Stream per tipi primitivi

Per facilitare l'utilizzo degli stream con i **tipi primitivi**, Java 8 introduce delle versioni specializzate della classe `Stream`.
Questo perché i tipi primitivi (`int` , `double` , `long`) **non possono essere gestiti direttamente da** `Stream<T>` **senza passare attraverso le classi wrapper** (`Integer` , `Double` , `Long`), **con conseguente overhead dovuto all'autoboxing.**

Le principali specializzazioni sono:

- `IntStream`

- `DoubleStream`
- `LongStream`

Queste classi mettono a disposizione metodi specifici per operazioni numeriche, come `sum()`, `average()`, `max()`, `min()`, che non sono presenti nello `Stream<T>` generico.

Perché servono le specializzazioni?

Supponiamo di voler sommare le calorie di una lista di oggetti `Dish`:

Java

```java
int calories = menu.stream()
    .map(d -> d.getCalories())
    .sum();   // NON compila
```

Questo codice **non compila** perché:

- il metodo `map()` **restituisce uno** `Stream<Integer>` **(quindi uno stream di oggetti wrapper),**
- `Stream<T>` **non possiede il metodo** `sum()`**.**

Per ottenere un flusso di valori primitivi `int`, è necessario usare il metodo specializzato `mapToInt()`:

Java

```java
int calories = menu.stream()
    .mapToInt(d -> d.getCalories())
    .sum();   // Compila e funziona
```

In questo caso:

- `mapToInt()` **restituisce un** `IntStream`**,**
- `IntStream` **possiede il metodo** `sum()`**,**
- **si evita il costo dell'**autoboxing**.**

> ⚠ **Considerazione importante**
>
> Le specializzazioni non servono solo per poter usare metodi come `sum()`, ma anche per:

- **migliorare le performance,**
- **ridurre la creazione di oggetti wrapper,**
- **rendere il codice più espressivo quando si lavora con valori numerici.**

In modo analogo esistono:

- `mapToLong()` → **restituisce un** `LongStream`
- `mapToDouble()` → **restituisce un** `DoubleStream`

Le specializzazioni rappresentano quindi:

- **un'estensione naturale del concetto di stream, pensata per lavorare in modo più efficiente e diretto con dati numerici primitivi.**

Operazione finale: `collect`

Il metodo `collect()` **rappresenta una delle operazioni finali più importanti degli stream.**
In forma più semplice:

- **consente di trasformare i dati presenti in uno** `Stream` **in una Collection o in una Map, quindi di "materializzare" il risultato della pipeline.**

Tuttavia, `collect()` non è soltanto un semplice meccanismo di copia:

- va interpretato come una **forma di riduzione** (concettualmente vicina a `reduce()`), perché gli elementi dello stream vengono:

1. **processati,**
2. **accumulati,**
3. **organizzati nella struttura finale.**

L'operazione di processing è formalmente definita dall'interfaccia `Collector`.

Nel caso più semplice, ad esempio:

Java

```java
.collect(Collectors.toList())
```

non viene applicata alcuna logica di trasformazione aggiuntiva:

- **gli elementi vengono semplicemente raccolti in una lista.**
  In questo scenario il `Collector` si limita ad accumulare i risultati senza ulteriori elaborazioni.

🔬 Riduzioni complesse e analogia con SQL

Il vero punto di forza di `collect()` emerge quando si definiscono **riduzioni più articolate**, come:

- **raggruppamenti,**
- **conteggi,**
- **aggregazioni,**
- **calcoli statistici.**

Queste operazioni sono concettualmente analoghe alle operazioni di **Group By** o alle funzioni di aggregazione in SQL.

La classe `Collectors`

La classe `Collectors` mette a disposizione numerosi metodi statici che forniscono logiche di accumulazione già pronte. I principali sono:

- `toList()` → **raccoglie gli elementi in una lista (nessuna elaborazione aggiuntiva).**
- `counting()` → **restituisce il numero di elementi presenti nello stream.**
- `groupingBy()` → **raggruppa gli elementi in base a una chiave.**
- `maxBy()` → **restituisce l'elemento massimo secondo un `Comparator`.**
- `minBy()` → **restituisce l'elemento minimo secondo un `Comparator`.**
- `summingInt()`, `summingDouble()`, `summingLong()` → **calcolano la somma.**
- `summarizingInt()`, `summarizingDouble()`, `summarizingLong()` → **producono statistiche complete (count, min, max, average, sum).**
- `averagingInt()`, `averagingDouble()`, `averagingLong()` → **calcolano la media.**
- `joining()` → **concatena gli elementi di uno stream in un'unica stringa.**
- `toMap()` → **accumula gli elementi in una mappa, specificando funzioni per chiavi e valori.**

È importante sottolineare che tutti questi metodi sono, in sostanza, **varianti più espressive e convenienti del metodo** `reduce()`.
Per questo motivo vanno interpretati come operazioni di riduzione specializzate, pensate per coprire i casi d'uso più frequenti in modo leggibile e dichiarativo.

> **In conclusione:**
>
> `collect()` rappresenta il punto in cui la pipeline termina e i dati, fino a quel momento elaborati in modo lazy, vengono effettivamente aggregati e resi disponibili in una struttura concreta.

Costruzione degli Stream

Finora abbiamo analizzato le operazioni che si possono applicare a uno stream; è quindi naturale chiedersi **come si costruisce uno stream**.

Uno stream può essere creato a partire da diverse sorgenti.
La costruzione rappresenta il punto iniziale della pipeline, **cioè la sorgente dei dati su cui verranno poi applicate le operazioni intermedie e finali.**

Stream a partire da una collezione

Il caso più comune è la costruzione di uno stream a partire da una **collezione già esistente** (ad esempio `List`, `Set`, ecc.).

Ogni collezione fornisce il metodo:

Java
```
stream()
```

e, in alternativa:

Java
```
parallelStream()
```

In questo modo si ottiene uno `Stream<T>` contenente gli elementi della collezione.

Stream a partire da dati espliciti

È possibile creare uno stream direttamente da un elenco di valori, utilizzando i metodi statici della classe `Stream`:

```java
Stream<String> s = Stream.of("a", "b", "c");
```

nalogamente esistono versioni specializzate per tipi primitivi:

- `IntStream`
- `DoubleStream`
- `LongStream`

Ad esempio:

```java
IntStream s = IntStream.of(1, 2, 3, 4);
```

Stream a partire da un array

Se i dati sono contenuti in un array, si può usare la classe `Arrays`:

```java
int[] numeri = {1, 2, 3, 4};
IntStream s = Arrays.stream(numeri);
```

Anche in questo caso esistono overload specifici per:

- `IntStream`
- `DoubleStream`
- `LongStream`

Se l'array contiene oggetti, il metodo restituisce uno `Stream<T>`.

Stream a partire da un file

È possibile creare uno stream leggendo dati da un file, **tramite i metodi statici della classe `Files`.**

Ad esempio, è possibile ottenere uno stream di righe di testo:

```java
Stream<String> lines = Files.lines(path);
```

Ogni elemento dello stream corrisponde a una riga del file.

Stream generati da funzione (stream infiniti)

È anche possibile costruire stream a partire da una **funzione generatrice:**

- **producendo sequenze potenzialmente infinite.**

In questi casi **lo stream non deriva da una struttura dati preesistente, ma viene generato dinamicamente sulla base di una funzione.**

Questa tipologia di stream richiede attenzione, perché **deve essere sempre accompagnata da operazioni come `limit()` per evitare elaborazioni infinite.**

Stream a partire dai dati

Un primo modo per costruire uno stream consiste nel creare **uno stream direttamente a partire da un insieme di valori espliciti.**

La classe `Stream` mette a disposizione il metodo statico `of()`:

- **che consente di fornire un numero variabile di elementi (da 0 a n):**
  - `static <T> Stream<T> of(T... values)`
  - `static <T> Stream<T> of(T t)`

Il metodo utilizza i **varargs:**

- **quindi è possibile passare uno o più elementi separati da virgola.**
  Lo stream risultante conterrà esattamente quei valori.
  Ad esempio:

```java
Stream<String> stream = Stream.of("Java 8 ", "Lambdas ", "In ", "Action");

stream.map(String::toUpperCase)
      .forEach(System.out::println);
```

In questo caso:

1. **lo stream viene creato a partire da quattro stringhe,**
2. **viene applicata una trasformazione (`map`) per convertirle in maiuscolo,**
3. **infine `forEach()` stampa ogni elemento.**

Stream vuoto

La classe `Stream` fornisce anche il metodo:

- `static <T> Stream<T> empty()`

che restituisce uno stream senza elementi.

Esempio:

```java
Stream<String> emptyStream = Stream.empty();
```

Questo è utile nei casi in cui si voglia restituire uno stream valido ma privo di dati, evitando di restituire `null`.

Concatenazione di stream

È inoltre possibile ottenere uno stream come **concatenazione di due stream esistenti**.

Per gli stream specializzati esistono metodi come:

- `static IntStream concat(IntStream a, IntStream b)`

In modo analogo, **anche la classe `Stream` fornisce una versione generica di `concat()`.**

La concatenazione crea un nuovo stream che contiene prima tutti gli elementi del primo stream e poi quelli del secondo, rispettandone l'ordine.

Stream a partire da un array

Un altro modo molto comune per costruire uno stream è partire da un array.

La classe `Arrays` mette a disposizione il metodo statico:

```java
```

```
stream()
```

che consente di trasformare un vettore (array) in uno stream.

- **Se l'array contiene oggetti, il metodo restituisce uno `Stream<T>`;**
- **se invece contiene tipi primitivi, vengono utilizzate le versioni specializzate (`IntStream`, `DoubleStream`, `LongStream`).**

Esempio con array di interi

Supponiamo di avere un array di `int`:

Java

```java
int[] numeri = {1, 2, 3, 4, 5};
```

Possiamo convertirlo in un `IntStream` e calcolare la somma:

Java

```java
int somma = Arrays.stream(numeri)
                  .sum();

System.out.println(somma);
```

In questo caso:

1. `Arrays.stream(numeri)` restituisce un `IntStream`,
2. il metodo `sum()` **(disponibile nelle specializzazioni) calcola la sommatoria,**
3. il risultato finale è un singolo valore `int`.

> ⚠ **Considerazione importante**
>
> L'uso delle specializzazioni (`IntStream`, `DoubleStream`, `LongStream`) è particolarmente utile quando si lavora con array di primitivi, perché:
>
> - **evita l'autoboxing,**
> - **migliora le performance,**
> - **consente l'uso diretto di metodi come `sum()`, `average()`, `min()`, `max()`.**

La costruzione di uno stream a partire da un array rappresenta quindi **un passaggio naturale quando si vuole applicare la logica dichiarativa degli stream a dati già memorizzati in forma vettoriale.**

Stream a partire da un file

Un'ulteriore modalità di costruzione di uno stream **consiste nel leggere i dati direttamente da un file.**

La classe `Files`, introdotta con Java NIO:

- **mette a disposizione diversi metodi statici che consentono di ottenere uno stream a partire dal contenuto di un file.**
  In questo caso, la sorgente della pipeline non è una collezione in memoria, ma il **file system**.

Poiché si tratta di operazioni di I/O, **questi metodi possono sollevare eccezioni (ad esempio `IOException`) e devono quindi essere gestiti opportunamente.**

Metodo `lines()`

Uno dei metodi più utilizzati è:

Java

```java
Files.lines(Path path, Charset cs)
```

Il metodo `lines()` restituisce uno `Stream<String>` in cui:

- **ogni elemento dello stream corrisponde a una riga del file,**
- **la lettura avviene in modo lazily, riga per riga.**

Esempio:

Java

```java
try (
    Stream<String> lines = Files.lines(Path.of("data.txt"),
                                       Charset.defaultCharset())
) {
    // uso dello stream
}
```

In questo esempio:

1. **viene aperto il file** `data.txt`**,**
2. **ogni riga diventa un elemento dello stream,**
3. **lo stream viene utilizzato all'interno di un blocco** `try-with-resources`**.**

ⓘ Perché usare il try-with-resources?

Lo stream ottenuto da `Files.lines()` è collegato a una risorsa esterna (il file).
È quindi necessario chiuderlo correttamente al termine dell'utilizzo.

Il costrutto `try-with-resources` **garantisce che lo stream venga chiuso automaticamente, evitando problemi legati alla gestione delle risorse.**